An implementation of Open-RMF in an industrial environment

Michele Belletti

2025
December

Contents

Part I

Introduction

This work presents a study on the new system, OPEN-RMF, which is still under active development but has reached a mature enough state to be deployed with robots at the Elettra Sincotrone Trieste facility.

Chapter 1

Background and motivation

Elettra Sincontrone Trieste[1] s a major research facility that primarily provides users access to two light sources: the synchrotron Elettra and the free electron laser Fermi. In addition to operating these sources, the facility actively researches new technologies and techniques to enhance its capabilities and operations. Many recent advancements are based on robotics, which, especially after the COVID-19 pandemic and with the rapid rise of large language models (LLMs), have enabled new ways for users to interact with the infrastructure, including robot-assisted information retrieval and package delivery services. Previously, the facility developed a global planner, D-Star (D-star based optimized trajectory planner)[12], within the ROS 1 framework to enable robot navigation across large-scale environments. However, as ROS 1 has reached end-of-life, the codebase must now be migrated to ROS 2. Additionally, the facility employs robots from multiple vendors, each specialized in different tasks and not designed to coordinate with one another. In an environment that shares resources such as narrow passages and lifts, this lack of interoperability creates significant coordination challenges. To address this issue, Open-RMF[2] has been developed in recent years as a multiplatform system for controlling and managing heterogeneous fleets of robots.

[1] https://www.elettra.eu/

[2] https://www.open-rmf.org/

Chapter 2

Research questions and objectives

The objectives of this thesis are as follows: to integrate a Rover Mini robot from the company RoverRobotics [1] into the ROS 2 Nav2 ecosystem, to port the D-Star global planner from ROS 1 to ROS 2,

Figure 2.1: RoverRobotics Rover Mini

incorporate the robot into the Open-RMF framework for centralized fleet management, compare its performance to the ROS 1-based navigation stack, and evaluate the suitability of the Open-RMF ecosystem for deployment at the Elettra Sincrotrone Trieste facility.

[1] https://roverrobotics.com/

Chapter 3

Scope and limitations

The principal limitation is the current lack of access to lift control systems, which prevents direct integration and operation by the robot fleet. Additionally, there is no automated system for opening and closing doors. While Open-RMF offers a high degree of customizability, and human-in-the-loop interactions can be configured—such as triggering a light or sending a notification when a robot approaches a door—this method requires a human to manually open the door or operate the lift. However, such an approach is not ideal for achieving full autonomy.

Part II

Literature Review

In this section, we provide a description of the key technologies used in this thesis. We begin by describing the Robot Operating System (ROS) and its functionality, followed by an overview of the navigation system employed, and a discussion of how the Open-RMF fleet management system operates.

Chapter 4

Overview of ROS1 and ROS2

The Robotics Operating System (ROS)[1] is an open-source collection of tools and libraries designed to support the development of robotics applications. Despite its name, ROS is not an operating system (OS) in the traditional sense, but rather a software development kit (SDK) that provides developers with a comprehensive set of tools for building robotic systems. It offers several features typically associated with an OS, including hardware abstraction, low-level device control, implementation of commonly used functionalities, inter-process communication, and package management. The ROS ecosystem is built upon the following core components:

- **Middleware communication layer (RMW)**: Built on the Data Distribution Service (DDS) standard, this layer enables nodes to exchange information using an anonymous publish/subscribe pattern. This design promotes fault isolation, separation of concerns, and modular interfaces. ROS also supports synchronous, Remote Procedure Call (RPC)-style communication between services. While the default DDS implementation is eProsima Fast DDS (`rmw_fastrtps_cpp`), this work uses Eclipse Cyclone DDS (`rmw_cyclonedds_cpp`) due to its improved compatibility with the Nav2 navigation stack.
- **Development tools**: ROS provides a suite of tools for managing and enhancing software development, including utilities for launching, introspection, debugging, visualization, plotting, logging, and playback.
- **Multi-machine support** ROS includes tools and libraries for retrieving, building, writing, and executing code across multiple computers and OS.
- **Community and governance**: ROS is supported by an active and extensive global community, coordinated primarily by Open Robotics[2], which serves as the central organization for its development and maintenance.

4.1 Structure of ROS

The official ROS documentation[3] provides comprehensive information about the system. All communication in ROS is handled through the **ROS Graph** , a peer-to-peer network of processes (called nodes) that execute concurrently and exchange

[1] https://www.ros.org/
[2] https://www.openrobotics.org/
[3] https://docs.ros.org/en/rolling/index.html

information with one another. Some components of this graph are described below. Note that some of the functionalities described are specific to ROS 2 and are not available in ROS 1:

- **Nodes**: A node is a modular process that performs a specific task. Nodes communicate with each other through topics, services, actions, and parameters. In ROS 2, nodes can be implemented as **LifecycleNode**, which provide a managed state machine with defined transitions for startup and shutdown, enabling more deterministic behavior. Nodes can also be composed within the same process using the

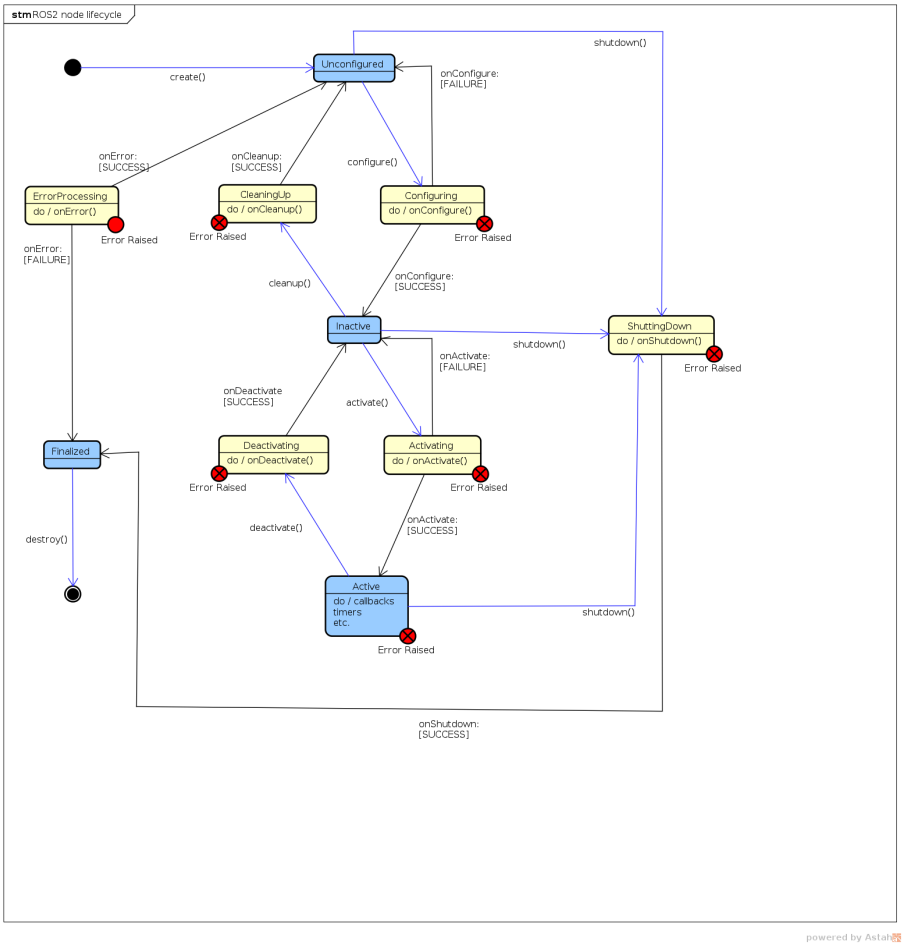

Figure 4.1: State Machine of LifecycleNode

Composition feature, which improves performance by bypassing middleware overhead[4].

- **Topics**: Topics are used for communication between nodes to share streams of messages in a publisher-subscriber structure.

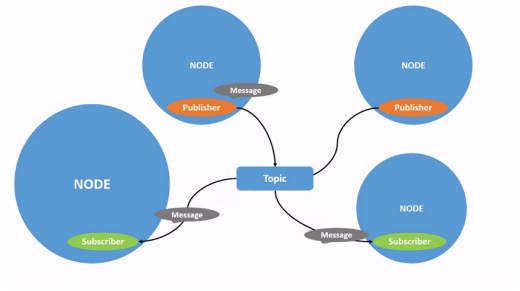

Figure 4.2: Topic process

- **Service**: Services are based on the request-response model. When a node with a service client sends a Request Message to a node with a service server, it receives a Response Message from that specific server.

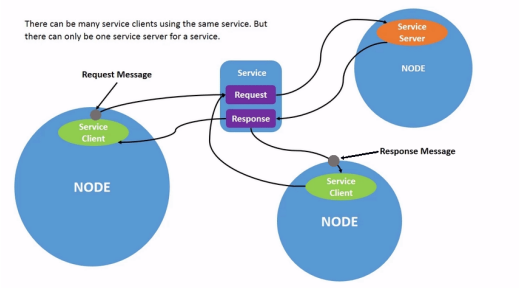

Figure 4.3: Service process

- **Parameters**: Parameters are configurations of a node that can be modified without changing the source code.
- **Actions**: Actions are designed to support long-running tasks that require feedback and the ability to preempt. They combine two services (goal and result) and a publisher-subscriber mechanism (feedback). A node sends a goal request and receives an acknowledgment, then starts publishing feedback messages while executing the task, and finally responds with the result.

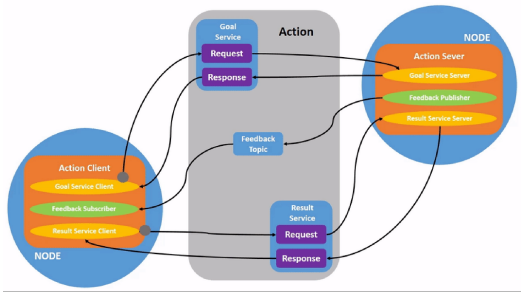

Figure 4.4: Action process

4.2 Tools of ROS

Now, I will describe the tools that are built into ROS for developing robotic systems:

- **Launch**: A file structure that allows the simultaneous start of multiple nodes with a set of parameters.

- **rqt**: A graphical user interface (GUI) tool for ROS. It includes a series of plugins that facilitate development, monitoring, and diagnostics of the ROS system.
- **rqt_console** A plugin for rqt that visualizes the logs generated by the ROS.
- **Rviz**: A 3D visualization tool for the Robot Operating System. It allows visualization of sensor data and state information from ROS.

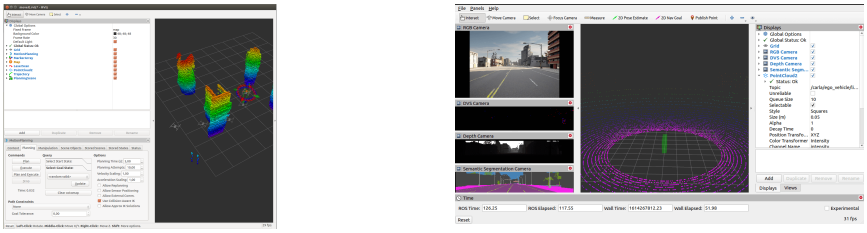

Figure 4.6: Examples of visualization of ROS data with Rviz

- **rosdep**: A command-line utility for installing system dependencies required by ROS packages.
- **sros2**: A security extension for ROS 2 that provides tools and instructions to apply security policies to DDS (Data Distribution Service).

Another important category of tools that are integrated with ROS is simulation environments. These environments allow developers to test their code on simulated robots under various conditions, which helps identify bugs that could potentially harm a real robot or its surroundings. Simulation tools provide realistic physical behavior for all components of a robotic system. According to the ROS documentation, two main simulation tools are commonly used: Gazebo and Webots. Among them, Gazebo is also maintained by Open Robotics and is widely adopted in many projects, including the one discussed in this paper. Gazebo offer:

- **ROS Integration**: A bridge that converts Protobuf messages to ROS messages, enabling communication between Gazebo and ROS.
- **Performance tools**: Features for distributed simulation across servers, dynamic asset loading, and adjustable time steps to optimize simulation performance.
- **Realistic simulation**: Support for a variety of sensors, including their noise models, within a 3D world rendered using OGRE 2.1[4] and powered by the DART5[5] physics engine.
- **Extensibility**: A modular design that supports plugins, allowing developers to run custom code that interacts directly with the simulation environment.

[4]https://wiki.ogre3d.org/Home
[5]https://dartsim.github.io/

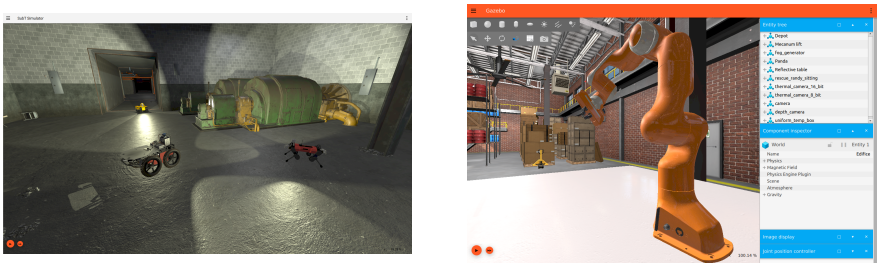

Figure 4.8: Examples of Gazebo

Chapter 5

Introduction to NAV2 and Google Cartographer

Multiple tools are available to enable a robot to move autonomously within an environment. One of the most complete systems is Nav2 [1] [7], the successor to the ROS 1 Navigation Stack.

5.1 Description of Nav2

Nav2 provides a comprehensive set of tools for perception, planning[9], control, localization, and visualization, allowing any robot to navigate autonomously and reliably, even in complex environments. It enables building a model of the environment from sensor data, dynamically calculating the optimal path to avoid obstacles, setting motor velocities, and executing navigation tasks using Behavior Trees.

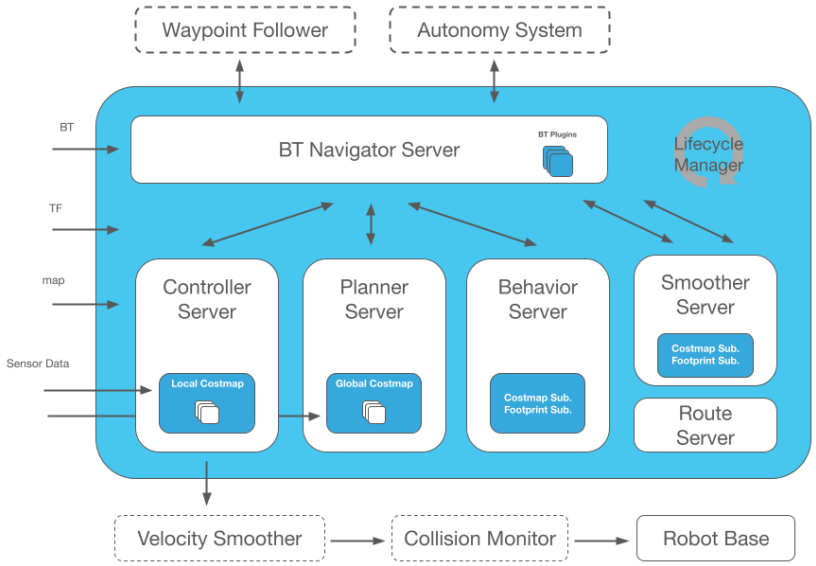

Figure 5.1: Nav2 architecture

Nav2 introduces several improvements over ROS 1, including:

[1] https://nav2.org/

- **Behavior trees**[1]: Nav2 uses Behavior Trees to structure and execute tasks in a human-readable and modular way. These trees enable the creation of complex systems and can be tested with various tools. The BehaviorTree.CPP[2] library is used to implement them, and it allows linking different trees for flexible behavior management.

- **nav2_util LifecycleNode**: This is a wrapper around the standard `rclcpp_lifecycle::Lifecyc` described earlier 4.1. It adds a bond connection to the Lifecycle Manager, which monitors the state of each node and can safely transition nodes through their lifecycle states in the event of a failure.

- **Action Server**: As described in 4.1, Nav2 uses action servers for Behavior Tree interactions, such as NavigationToPose (navigate to a specific location) and NavigationThroughPoses (navigate through multiple waypoints).

Nav2 is composed of several key nodes that work together to control the robot:

- **Controller Server**: Handles movement requests. It uses plugin-based modules such as a progress checker (to verify if the robot is making progress), goal checker (to determine if the robot has reached its destination), and a controller (to generate velocity commands using the **local costmap** while avoiding collisions).

- **Planner Server**: Computes the global path to the destination using sensor data and a plugin-based global planner loaded with the **global costmap**.

- **BT Navigator Server**: Implements the NavigateToPose, NavigateThroughPoses, and other task interfaces. It is a Behavior Tree-based implementation of navigation that is intended to allow for flexibility in the navigation task and provide a way to easily specify complex robot behaviors.

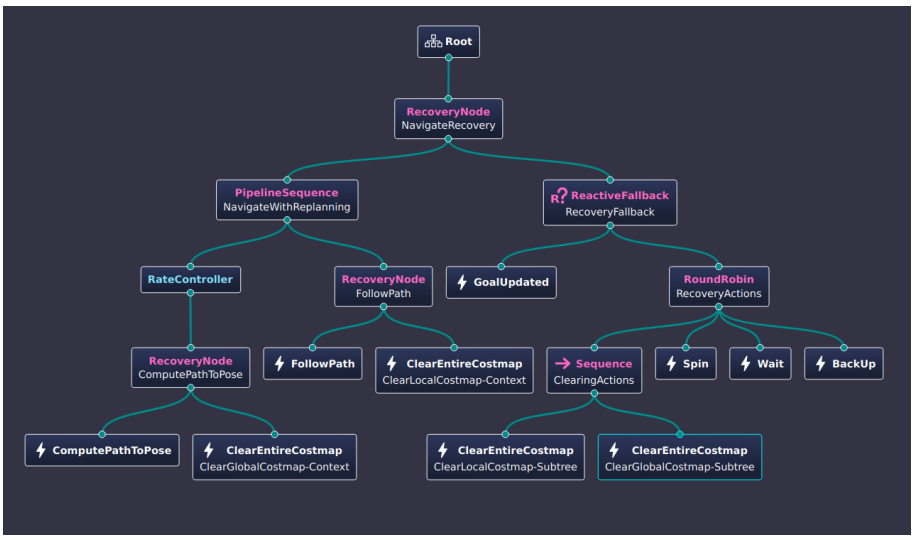

Figure 5.2: Humble Nav2 BT navigation `navigate_to_pose_w_replanning_and_recovery.xml` loaded with Groot2

[2]https://www.behaviortree.dev/

- **Smoother Server**: Smooth the path minimizing sharp turns and rapid rotations.
- **Behavior Server**: Handles specific behavior actions like spinning, backing up, and waiting.
- **Waypoint Follower**: An example of orchestrated control, this node takes a list of waypoints and uses the NavigateToPose action server to move between them, optionally performing custom tasks (e.g., taking a photo) at each waypoint through plugins. It can be replaced by any **Autonomy System**
- **Route Server**: Calculates routes through a predefined navigation graph instead of using free-space planning.
- **Velocity Smoother**: Smooths velocity, acceleration, and deadband signals sent to the robot to ensure safe and smooth motion.
- **Costmap** is a representation of the environment using a regular 2D grid of cells with a cost (unknown, free, occupied, or inflated) and is used for generate the global plan or to compute local control efforts.

The next things that Nav2 relies on for controlling the navigation are:

- **state estimation** of the robot that generate a suitable TF tree, a tree structure of coordinate frames generated using the transform library[2], based on the REP 105, that is should provide this structure: `map` → `odom` → `base_link`. According to the REP 105 they are defines as:
  - `map` : A world-fixed frame that may change in discrete jumps. It's useful for long-term global reference but not suitable for short-term localization.
  - `odom` : A continuous world-fixed frame that may drift over time. It provides short-term local reference.
  - `base_link`: A robot fix frame.

  The `odom` → `base_link` link can be estimated using various sensors. For improved accuracy, we use the `robot_localization` package, which provides nonlinear state estimation using Kalman Filter [8].
  The `map` → `odom` link can be generate with a global positioning system like Simultaneous Localization And Mapping (SLAM) method or using the one provided by Nav2, Adaptive Monte-Carlo Localization (AMCL).
- **map** can either be preloaded or built dynamically using a SLAM method. One SLAM method systems is Google Cartographer, which will be described in the next section. It is also the SLAM method that was used to generate the map for navigation.

5.1.1 Cartographer

Cartographer[3] is a SLAM method that operates using two main subsystems: a **Local SLAM** (called **Frontend**) that build a series of locally consistent **submaps** from successive scans and the pose extrapolator but with the possibility that it can

drift during acquisition and a **Global SLAM** (called **Backend**) reduces drift by performing loop closure—detecting when the robot has returned to a previously visited location. It uses scan matching against submaps and applies global optimization to align all submaps consistently and build a globally accurate map. Cartographer is capable of achieving mapping accuracy within 2–3 cm, but it requires extensive parameter tuning to reach optimal performance.

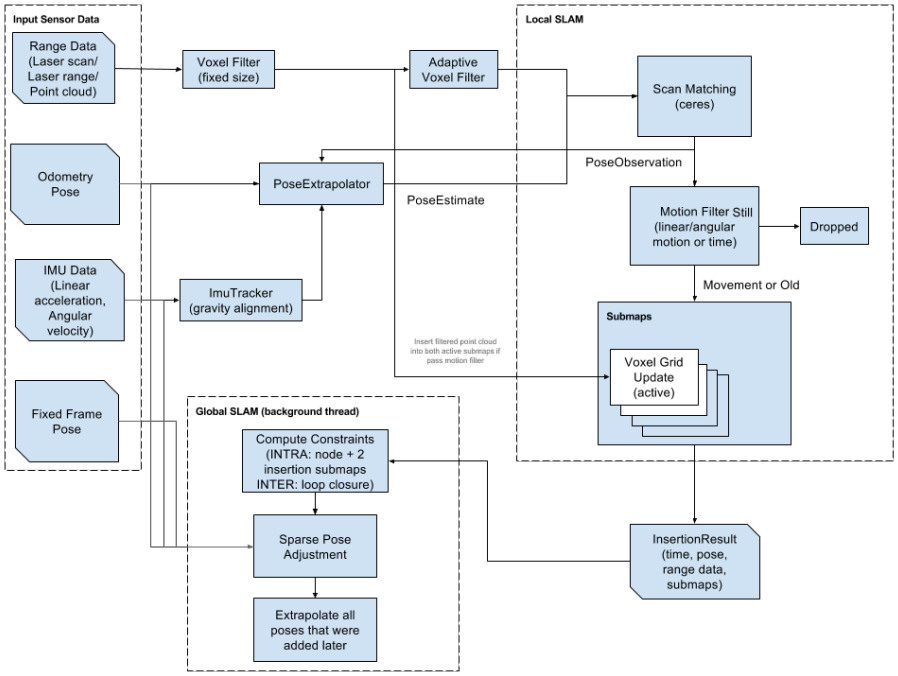

Figure 5.3: Cartographer Algorithm

Chapter 6

Review of related work on porting navigation stack from ROS 1 to ROS 2

The widespread adoption of the Robot Operating System (ROS) has introduced new use cases that were not originally considered during its development. Initially created as an academic project, ROS encountered limitations as users began applying it to more complex scenarios such as fleet management, deployment on embedded boards and microcontrollers, real-time systems, unreliable networks, and industrial applications. These emerging requirements highlighted architectural constraints in ROS 1, ultimately necessitating a complete redesign, which led to the development of ROS 2. Comprehensive documentation for ROS 2 is available at https://design.ros2.org/ . Many companies have adopted the ROS 2 ecosystem, noting improvements in development workflows[6] and in specific subsystems like Nav2[9]. New initiatives, such as micro-ROS[1], aim to bring ROS 2 capabilities to microcontrollers. Additionally, features like Node Composition offer improved performance optimization[4] while sros2 allow for a better security[11]. As outlined in these references, ROS 2 offers substantial advantages over ROS 1, making it the preferred framework for modern robotic applications. With this in mind, we will migrate the global planner using the official ROS 2 documentation[2] and various secondary sources [3].

[1]https://micro.ros.org/

[2]https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html

[3]https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html

Chapter 7

Overview of open source fleet management systems, particularly Open-RMF

Currently, several fleet management systems are available:

- **Isaac Mission Dispatch**[1]: A microservice-enabled fleet management system developed by NVIDIA that allows users to submit missions to multiple robots and monitor both robot and mission statuses. It provides connectivity between robots and the fleet management system but does not handle logistics such as task allocation or conflict resolution (e.g., intersecting robot paths). The implementation relies on the VDA5050 protocol, an industry standard for communication between cloud control services and mobile robots, and uses MQTT, a lightweight, publish-subscribe network protocol designed for devices with limited resources and bandwidth.
- **ROOSTER Fleet Management**[2]:An open-source ROS 1-based project that supports heterogeneous fleet management with task allocation, scheduling, and routing functionalities.
- **Open Robotics Middleware Framework (Open-RMF)**[3]: A free, open-source, and modular system that facilitates interoperability between multiple robot fleets and physical infrastructure such as doors, elevators, and building management systems.
- **Robofleet**[4][10]: An open-source system based on ROS 1 that supports inter-robot communication, remote monitoring, and remote tasking for heterogeneous fleets of ROS-enabled service robots. It is specifically designed to handle network variability and to incorporate security control features.

Other frameworks provide the infrastructure needed to initiate fleet control:

- **Transitive Robotics**[5]: An open-source framework for full-stack robotics, designed to simplify the development of capabilities that connect robots, cloud

[1]https://github.com/NVIDIA-ISAAC/isaac_mission_dispatch
[2]https://github.com/ROOSTER-fleet-management
[3]https://www.open-rmf.org/
[4]https://github.com/ut-amrl/robofleet
[5]https://transitiverobotics.com/

services, and web front-ends through a dynamically updating shared data model. It addresses common challenges in building full-stack robotic applications, such as unreliable networks, intermittently offline robots, and cross-device versioning.

- **Robot Web Tools**[6]: A collection of tools that enable communication with remote robots via web technologies, facilitating remote monitoring and interaction.

Given that Open-RMF offers control over multiple fleets, enables structured interaction between them to achieve optimal outcomes, provides a comprehensive toolset for system development, is based on ROS 2, and is open source, it will be used as the foundation for our fleet management architecture.

7.1 Open-RMF description

Open-RMF is a flexible and modular toolkit built on ROS 2 that facilitates interaction between heterogeneous robot fleets. It employs standardized protocols to enable communication with infrastructure, environments, and automation systems, optimizing the utilization of critical shared resources such as elevators, doors, narrow corridors, and other robots. By coordinating access to these resources, Open-RMF introduces system-level intelligence that helps prevent conflicts and ensures proper task and resource allocation. Figure 7.1 illustrates the architecture and operational workflow of Open-RMF.

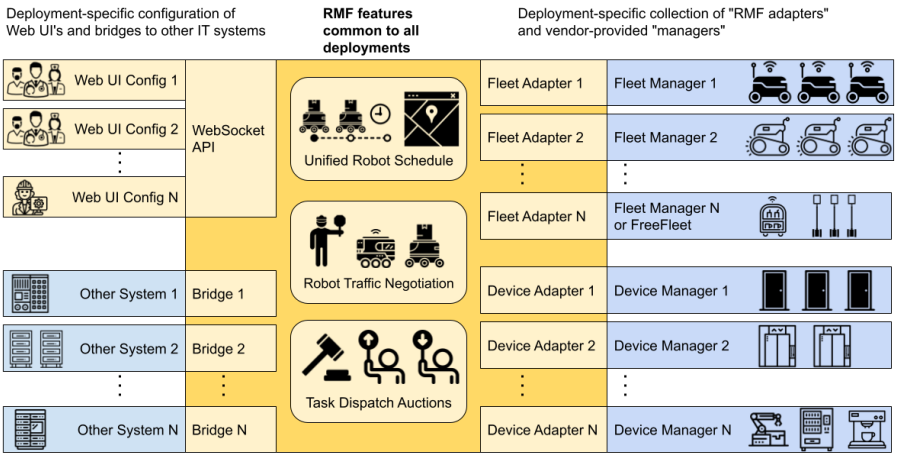

Figure 7.1: Open-RMF Structure

The fleets that can be controlled through Open-RMF can be categorized into three types:

- *Full Control*: Fleets of this type provide full status updates and allow complete control over path planning and execution.
- *Traffic Light*: These fleets also provide full status updates, but only allow limited control, such as pausing and resuming operations. Open-RMF has control over certain behaviors, but not over the specific paths.

[6]https://robotwebtools.github.io/

- *Read Only*: These fleets only provide status updates and cannot be controlled. Only one read-only fleet can exist in the system at any given time, as the presence of multiple read-only fleets would make it nearly impossible to avoid deadlocks or resource conflicts.

If a fleet lacks has *No Interface*, it can lead to deadlocks in resource allocation and is incompatible with an RMF system.
The core of Open-RMF, known as `rmf_core`, is composed of several modular packages, each serving a distinct role within the system:

- `rmf_traffic`: Provides core scheduling and traffic management capabilities. It is based on a platform-agnostic **Traffic Schedule Database** and includes two main mechanisms for traffic deconfliction:

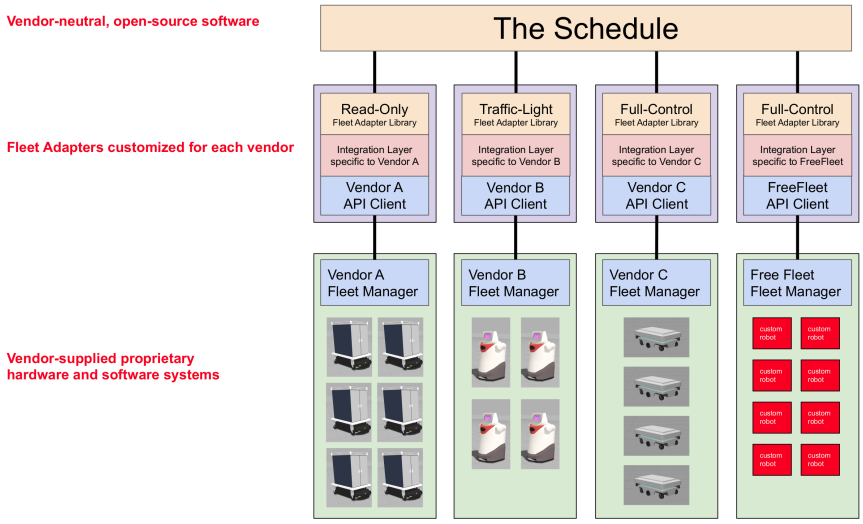

Figure 7.2: Open-RMF Schedule structure

  - *Prevention*: When changes occur in the database (e.g., route cancellations), fleet managers can proactively replan routes to avoid conflicts.
  - *Negotiation*: When a potential conflict is detected between two or more schedule participants, a conflict notice is issued to the relevant fleet managers. Each manager responds with a preferred path that accommodates other agents, and an external arbitration module selects the optimal path.

  Conflict avoidance for Automated Guided Vehicles (AGVs) is implemented using a time-dependent extension to A* search. This algorithm considers both spatial and temporal dimensions, allowing it to generate conflict-free paths by accounting for the future movements of other agents. Support for Autonomous Mobile Robots (AMRs) is under active development.

- `rmf_traffic_ros2`: Provides the necessary ROS 2 interfaces for integrating rmf_traffic into ROS 2-based systems.

- `rmf_task`: Contains APIs and base classes for defining and managing tasks in RMF. The framework natively supports three types of task requests:

  - *Clean*: For robots capable of cleaning floor spaces.
  - *Delivery*: For robots tasked with transporting items between locations.
  - *Loop*: For robots required to repeatedly navigate between two or more points.

- `rmf_dispatcher_node`: Contains the logic responsible for task allocation and management across multi-fleet systems. When a user submits a new task request, the dispatcher intelligently assigns it to the most suitable robot by querying each fleet adapter capable of performing the task. These adapters respond with bids indicating which robot is best suited, and RMF uses this bidding mechanism to determine the optimal assignment, as illustrated in Figure 7.3.

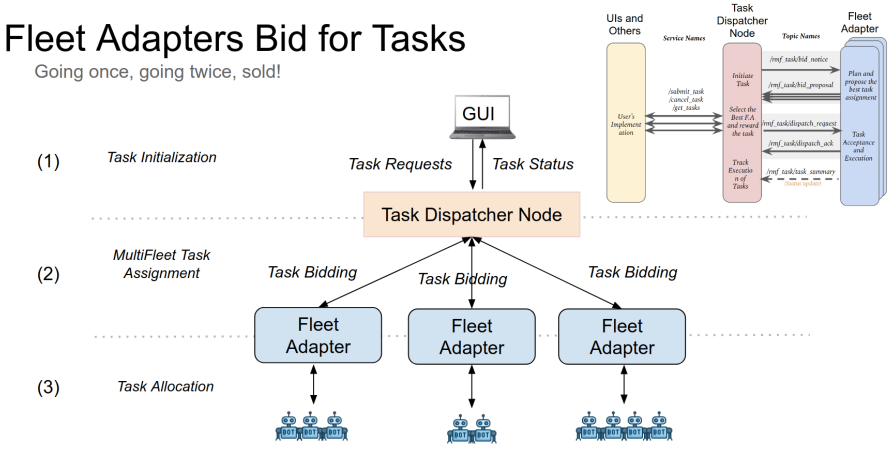

Figure 7.3: Open-RMF Bid Task

- `rmf_battery`: Provides battery consumption estimation models for different robot activities, aiding in energy-aware task planning.
- `rmf_ros2`: Supplies ROS 2 adapters, nodes, and Python bindings to integrate the RMF core components into ROS 2 applications.
- `rmf_utils`: A collection of utility functions and tools used across various RMF packages.

The following tools are available to support development with Open-RMF:

- **RMF demos**: A collection of demonstration packages showcasing the capabilities of Open-RMF. These serve as a useful starting point for development and experimentation.
- **Traffic Editor**: A graphical map editor used to define traffic patterns and interaction points (e.g., doors, lifts) that RMF uses for traffic management. It also supports the generation of Gazebo simulation worlds based on the designed maps.

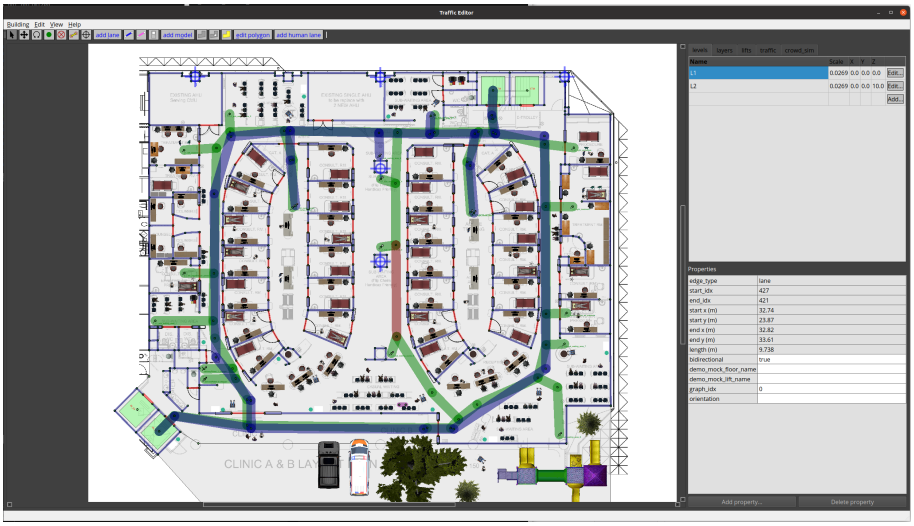

Figure 7.4: Traffic Editor

- **Free Fleet**: An open-source fleet adapter that enables the integration of standalone mobile robots into Open-RMF, allowing the creation of heterogeneous fleets.

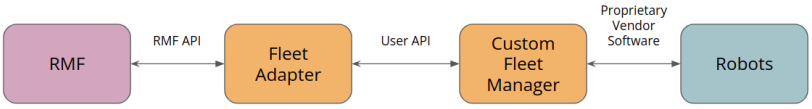

Figure 7.5: Fleet Adapter

- **RMF Schedule Visualizer**: An RViz plugin that provides visualization and monitoring of the RMF schedule, enabling users to observe and interact with traffic and task execution in real time.
- **RMF Web UI** : A web-based application for visualization and control of the Robotic Middleware Framework (`rmf_core`). It provides a user-friendly interface for monitoring system state and interacting with fleet operations.

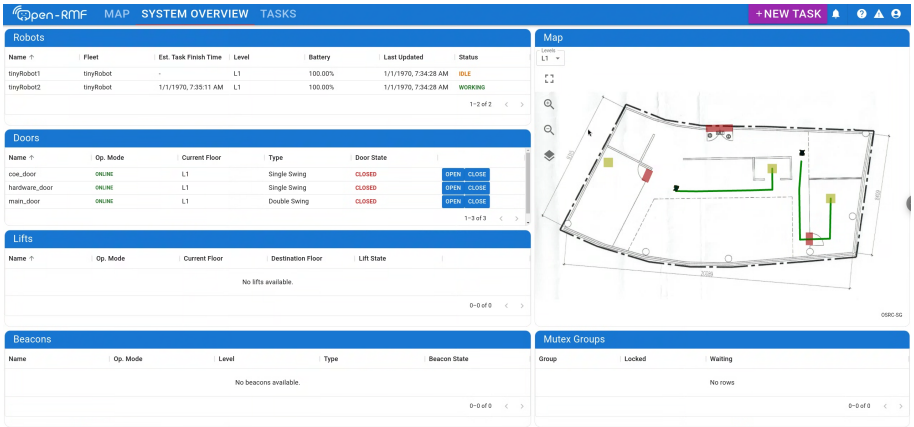

Figure 7.6: RMF Web

- **RMF Simulation**: A set of Gazebo plugins designed to simulate various aspects of building infrastructure—such as doors, lifts, and crowds—as well as robot behavior, including navigation and interactions with workcells.
- **Simulation assets**: Freely available 3D models and environment assets used to support simulation development and testing in Gazebo.

Part III

Methodology

In this part we will discuss all the procedure that took place from building our testing platform, based on the Rover Mini and the sensors used. Setting up the Nav2 navigation stack. The procedure used to port the Global planner from ROS 1 to ROS 2. How we integrate the system and the software that is used to setup the Open-RMF system.

Chapter 8

Description of research platform and hardware used

The robot is a 4 wheel drive (WD) skid-steering (see Figure 2.1)that has a maximum speed of 12.87 km/h with a payload of 45,36 kg. The company offer the code to control it and also to simulate on github[1]. The code is steel tested only for ROS 2 Humble, this is going to be our version of ROS 2 that we are going to use on the Rover.
For controlling it, the rover give a USB terminal for connecting to a PC or single board computer. For this purpose we are using a Mini PC where there is all the control of all the navigation based on Nav2. The Mini PC is a Ryzen 7 qbvqg as in picture
For Nav2

[1]https://github.com/RoverRobotics/roverrobotics_ros2

Chapter 9

Steps taken to port the navigation stack from ROS 1 to ROS 2

Chapter 10

Hardware adaptation process from Rover Zero to Rover Mini

Chapter 11

Integration of the navigation stack with Open-RMF

Chapter 12

Testing and evaluation of the system

Part IV

Results and Analysis

12.1 Presentation of results from testing and evaluation

12.2 Comparison of system performance before and after the porting process

12.3 Discussion on the challenges encountered and how they were addressed

Part V

Conclusion and Future Work

Part VI

Appendices

today the tool for converting the data are https://www.ros.org/blog/noetic-eol/
implement also planning benchmark
describe and use composition in nav2 to obtain better performance [5]

Inoltre per gestire il traffico sulla rete utilizzeremo zenoh (Zero Overhead Network Protocol) ed il bridge per ros2 zenoh-bridge-ros2dds che è pub/sub/query protocol unifying data in motion, data at rest and computations. It elegantly blends traditional pub/sub with geo distributed storage, queries and computations, while retaining a level of time and space efficiency that is well beyond any of the mainstream stacks. data liberator protocol. Zenoh liberates data in several dimensions.

with chatgpt possiamo gestire il robot in due modi: si ferma ed esegue l'azione non communicando con rmf (la linea rimane bloccata dal suo passaggio)

Bibliography

[1] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. July 2018. DOI: 10.1201/9780429489105. URL: http://dx.doi.org/10.1201/9780429489105.

[2] Tully Foote. "tf: The transform library". In: *Technologies for Practical Robot Applications (TePRA), 2013 IEEE International Conference on*. Open-Source Software workshop. 2013, pp. 1–6. DOI: 10.1109/TePRA.2013.6556373.

[3] Wolfgang Hess et al. "Real-Time Loop Closure in 2D LIDAR SLAM". In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278.

[4] Steve Macenski et al. *Impact of ROS 2 Node Composition in Robotic Systems*. 2023. arXiv: 2305.09933 [cs.RO]. URL: https://arxiv.org/abs/2305.09933.

[5] Steve Macenski et al. "Impact of ROS 2 Node Composition in Robotic Systems". In: *IEEE Robotics and Automation Letters* 8.7 (2023), pp. 3996–4003. DOI: 10.1109/LRA.2023.3279614.

[6] Steven Macenski et al. "Robot Operating System 2: Design, architecture, and uses in the wild". In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

[7] Steven Macenski et al. "The Marathon 2: A Navigation System". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.

[8] T. Moore and D. Stouch. "A Generalized Extended Kalman Filter Implementation for the Robot Operating System". In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, 2014.

[9] DV Lu A. Merzlyakov M. Ferguson S. Macenski T. Moore. "From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2". In: *Robotics and Autonomous Systems* (2023).

[10] Kavan Singh Sikand et al. *Robofleet: Open Source Communication and Management for Fleets of Autonomous Robots*. 2021. arXiv: 2103.06993 [cs.RO]. URL: https://arxiv.org/abs/2103.06993.

[11] Victor Mayoral Vilches et al. *SROS2: Usable Cyber Security Tools for ROS 2*. 2022. arXiv: 2208.02615 [cs.CR]. URL: https://arxiv.org/abs/2208.02615.

[12] Andrey Vukolov et al. “Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis”. In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.